Disaster Evacuation: Most Efficient Route Finder

Algorithmic Approaches to Dynamic Geospatial Routing

Presented by:

Ayush Dwivedi (RA2411003030121)

Ayaan Mirza Baig (RA2411003030133)

Pratik Raj (RA2411003030150)

The Challenge of Emergency Egress

Natural and anthropogenic disasters demand rapid, large-scale population displacement.

Static evacuation plans fail catastrophically under dynamic constraints.

Requirement: A real-time, computationally efficient routing mechanism adapting to rapidly changing spatial telemetry.

Dynamic Algorithmic Routing Engine

- Implement a routing system built on graph theory paradigms.
- Continuously ingest real-time geospatial data to update traversable topologies.
- Determine globally optimal escape vectors using heuristic-based pathfinding algorithms.

Topological Abstraction

Mathematical Modeling of the Evacuation Graph

We model the physical map as a weighted, directed graph $G=(V,E)$.

V (Vertices)

Intersections, safe zones, triage centers, and hazard epicenters.

E (Edges)

Traversable road segments connecting vertices.

W (Weights)

A dynamic cost function associated with traversal latency and environmental risk probability.

Dynamic Cost Function

Modulating Edge Weights in Real-Time

Edge weights are dynamically modulated scalars: **w(u,v) = alpha*d + beta*t + gamma*r**

d

Euclidean or Haversine distance.

t

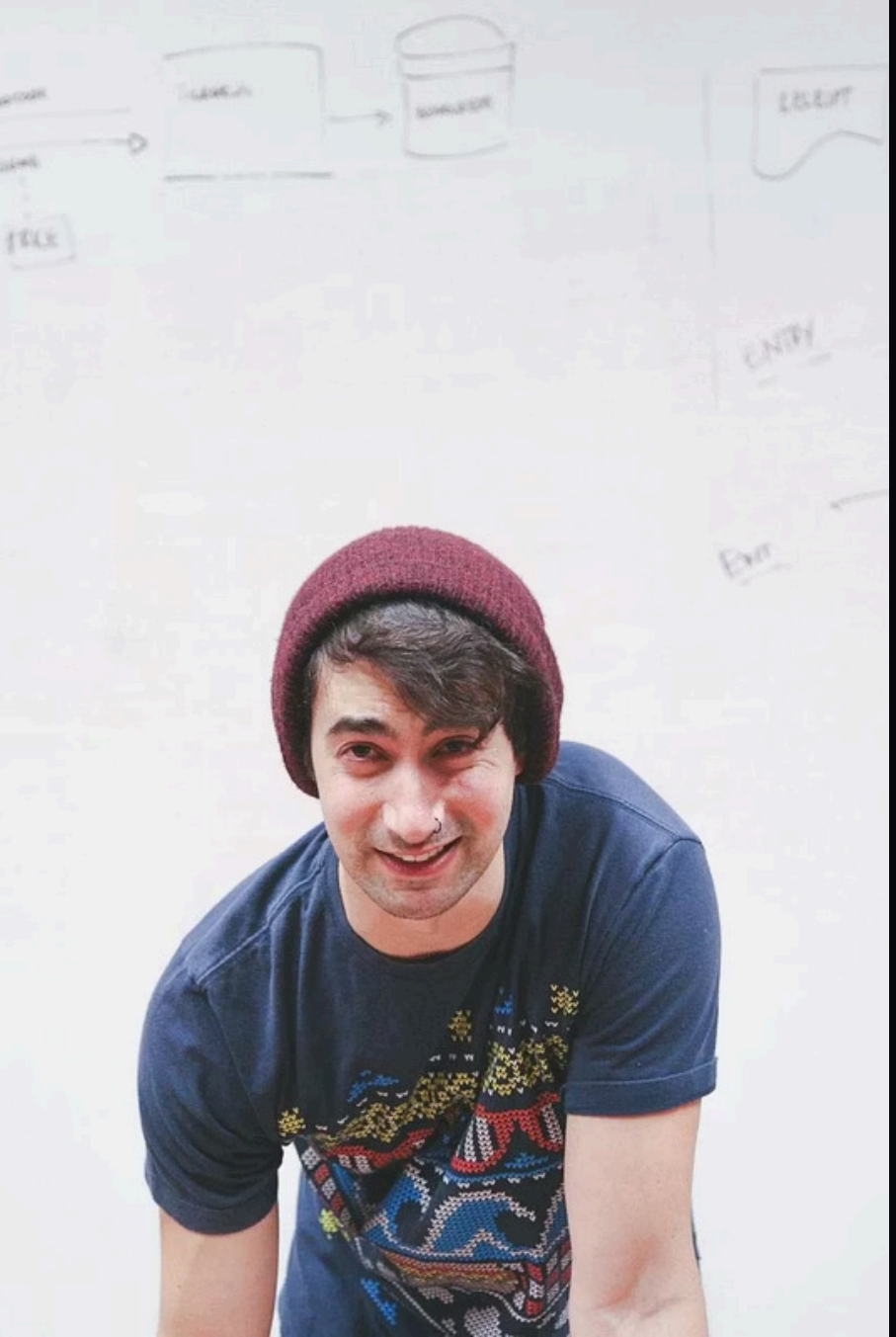
Traffic congestion density (latency).

r

Disaster proximity/hazard factor.

alpha, beta, gamma

Context-dependent tunable hyperparameters.



Uniform-Cost Search

The Baseline Approach - Dijkstra Algorithm

- Guarantees shortest path in a graph with non-negative edge weights.
- Uses a Greedy approach with a Min-Priority Queue.
- Time Complexity: $O(|E| + |V| \log |V|)$ with a Fibonacci heap.
- Drawback: Explores uniformly in all directions - expensive for real-world geospatial networks.

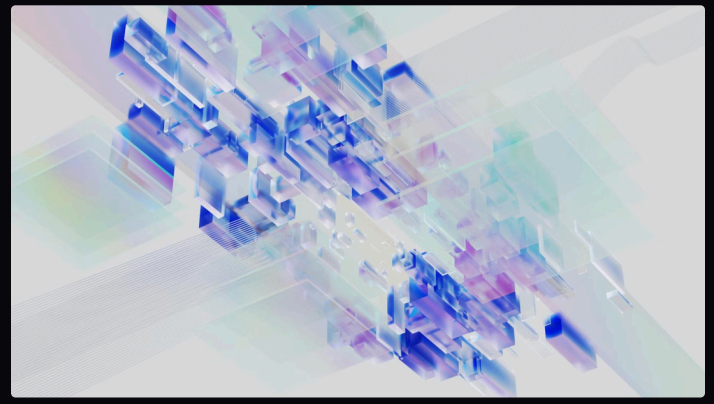
Heuristic-Driven Convergence

The Optimization - A* Search Algorithm

Integrates informed search with Dijkstra's uniform-cost mechanics.

Evaluates nodes using: **$f(n) = g(n) + h(n)$**

- $g(n)$: Exact path cost from source to node n.
- $h(n)$: Admissible heuristic estimating cost to target safe zone.
- Rapidly prunes non-optimal subtrees to minimize node expansion.



Maintaining Admissibility

Heuristic Function Design

h(n) must never overestimate the true cost to the safe zone.

Haversine Distance

Ideal for GPS coordinate tracking on Earth's spherical geometry.

Manhattan Distance

O(1) computation for grid-based urban block topologies.

Injecting hazard vectors into the heuristic repels routes away from expanding disaster zones.

Adapting to Topological Decay

Handling Dynamic Edge Invalidation

Disasters cause instantaneous edge invalidation (flooded street = edge deletion).

Recomputing A* from scratch every millisecond is inefficient.

Solution: D* Lite and Lifelong Planning A* (LPA*) algorithms.

These reuse previous search trees for efficient computation under continuous dynamic alterations.

Choosing the Right Tool

Comparative Algorithmic Analysis

Bellman-Ford

O(|V||E|). Handles negative weights but computationally heavy.

Floyd-Warshall

O(|V|^3) all-pairs shortest path. Good for peacetime pre-computation but not real-time.

A* / D* Lite

Optimal intersection of speed and adaptability for single-source real-time routing.

System Architecture Pipeline

From Sensors to Safety

Data Ingestion Layer

Asynchronous APIs fetching live telemetry (traffic APIs, IoT weather/seismic sensors).

Processing Engine

Cloud-based microservices executing the graph algorithms in memory.

Client Interface

Mobile application providing turn-by-turn navigation, receiving WebSocket streams for instant rerouting commands.

Core Logic Implementation

*A Pseudocode Snippet**

```
def a_star_evacuation(graph, start, safe_zone):
 open_set = PriorityQueue()
 open_set.put(start, 0)
 g_score = {node: infinity for node in graph}
 g_score[start] = 0

 while not open_set.empty():
 current = open_set.get()

 if current == safe_zone:
 return reconstruct_path(current)

 for neighbor in graph.neighbors(current):
 tentative_g = g_score[current] + weight(current, neighbor)

 if tentative_g < g_score[neighbor]:
 g_score[neighbor] = tentative_g
 f_score = tentative_g + heuristic(neighbor, safe_zone)
 open_set.put(neighbor, f_score)
```

Hands-On Algorithmic Walkthrough

Mental Exercise: Dynamic Rerouting

01

Scenario

Evacuating from Node A to Safe Zone D.

02

Path 1

A $\rightarrow$ B $\rightarrow$ D (Base Cost: 10).

03

Path 2

A $\rightarrow$ C $\rightarrow$ D (Base Cost: 15).

04

Event

A fire breaks out near node B. The hazard coefficient γ spikes.

05

Exercise

The dynamic cost $w(A, B)$ recalculates to 50. The open-set queue dynamically re-prioritizes. Path 2 is now extracted as the optimal route, effectively steering the user away from the hazard in real-time.

Future Enhancements

Scaling the Infrastructure

IoT Integration

Interfacing with smart city grids to automatically override traffic signals, creating uninterrupted green-light evacuation corridors.

Machine Learning

Utilizing Graph Neural Networks (GNNs) to predict hazard propagation trajectories before they occur.

Multi-Agent Systems

Implementing load-balancing algorithms to prevent thousands of users from being routed to the exact same "optimal" street, thus creating artificial congestion.

Thank You

Conclusion

Thank you for your time and attention.